iPRA: Smaller Checkers and Limits of Algebraic Simplification

Complete presentations, reproducible measurements, and a finite-parameter obstruction

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

This investigation obtains a smaller implementation family and a new obstruction theorem, but does not establish the requested single axiomatization below every current checker-size minimum. A complete checker for the inherited intuitionistic arithmetic presentation uses 276 normalized Python tokens and 305 AST nodes, improving the current 277-token and 311-node minima. A second complete implementation uses 652 normalized bytes, improving the current 672-byte minimum. These are different programs. Their maximum-compression gzip sizes are 398 and 394 bytes, respectively; compression is reported only as supplementary information.

The principal new mathematical result concerns the attempted removal of numerical cases. The countable structure of eventually constant sequences satisfies all primitive-recursive defining equations, constructor separation, every literal-equation induction instance, and even all universal Horn arithmetic laws true in the standard natural numbers. Nevertheless every finite parameter expansion of this structure has a nonidentity involution. A definable bi-interpretation would transfer that involution faithfully to an iPRA model, contradicting its definable linear order. This excludes ordinary bi-interpretability even with changed domains, definable quotients, and finite parameters for the specified class of proposed reductions.

We also give a complete alternative progressive-binder presentation of iPRA, explain why it has not produced a smaller checker, and rederive two inherited obstructions to tempting simplifications. Full inherited arithmetic and logical bridges are included with attribution. The proofs are written mathematical proofs, supported by executable checker tests; no Lean/Rocq formalization or absolute minimality theorem is claimed.

Outcome relative to the request. The implementation family improves all three observed coordinates separately. No supplied program beats all three at once, and the new mathematical results do not establish the requested significant positive simplification. The report records the strongest verified results rather than naming an unproved construction a breakthrough.

Contents

1 What was compared and what improved **4**
1.1 Measurement protocol . . . 4
1.2 New and inherited contributions . . . 5

2 The reference theory and the exact implemented presentation **5**
2.1 Which iPRA is meant . . . 5
2.2 Numerical syntax and all numerical axioms . . . 5
2.3 Logical syntax and all inference rules . . . 6
2.4 Recovering all intuitionistic logic . . . 7
2.5 Bi-interpretability, rather than just agreement on numerals . . . 7

3 What the checker changes actually do **8**
3.1 Persistent contexts and iterative continuations . . . 8
3.2 Why the byte refactorings are sound . . . 9
3.3 Syntax, freshness, and arithmetic packets . . . 9

4 Verification, files, and the remaining target **10**

5 A finite-parameter obstruction to removing numerical cases **12**
5.1 The ordinary theory under consideration . . . 12
5.2 A countable model of all literal-equation induction . . . 12
5.3 Finite parameters cannot remove the symmetry . . . 13
5.4 Why bi-interpretability cannot conceal this symmetry . . . 14
5.5 All true universal Horn laws still do not suffice . . . 15
5.6 Exact application to the progressive native calculus . . . 15

6 A progressive binder presentation of iPRA **17**
6.1 Numerical syntax and complete rule inventory . . . 17
6.2 Elementary derived operations . . . 18
6.3 Gates and selectors on arbitrary formulas . . . 18
6.4 All intuitionistic logical connectives . . . 19
6.5 Whole-domain bi-interpretability . . . 21

7 Why merging the initializer into the update loses multivariate power **22**
7.1 The two candidate operations . . . 22
7.2 The single-coordinate majorant invariant . . . 23
7.3 Consequences and exact scope . . . 23

8 Why atomic case introduction cannot simply be removed **24**

Attribution and the full inherited equivalence proof **26**

A Complementary introductions with one local assumption **26**
A.1 Complete native syntax and rule inventory . . . 26
A.2 Exact equivalence and the unused-witness construction . . . 27
A.3 Induction and numerical cases separate cleanly . . . 28
A.4 The entire ordinary logical interface . . . 29
A.5 Whole-domain realizations and both formula composites . . . 30
A.6 What the theorem does and does not claim . . . 31

B Internal scalar predecessor bootstrap **31**
B.1 Elementary terms and two finite cycles . . . 31
B.2 A ramp with a global scaling identity . . . 32
B.3 A comparator from scalar iteration . . . 33
B.4 Saturated iteration gives predecessor . . . 34
B.5 Dependency boundary . . . 35

C From literal-equation induction to full intuitionistic PRA **35**
C.1 The exact hypotheses . . . 35
C.2 Constructor separation and two elementary tests . . . 36
C.3 Four elementary uses of equation induction . . . 37
C.4 An equation invariant that reflects actual equality . . . 38
C.5 Compilation of all quantifier-free formulas . . . 39
C.6 Recovery of induction and the full theory . . . 40

D Pairing, primitive recursion, and internal numerical composites **41**
D.1 Pairing from a bounded square-root search . . . 41
D.2 Compiling each primitive recursion . . . 42
D.3 Why both numerical round trips are internal . . . 42

E Complete measured programs **43**
E.1 Iterative checker with immutable contexts . . . 43
E.2 Compact byte profile . . . 44

1 What was compared and what improved

The source snapshot is ProveIt commit `0300a290bedee5019c974291850b798378438b2e`. Its `docs/drafts` directory contains sixteen direct ZIP archives. The census follows embedded ZIPs as well: there are 22 distinct archive contents, 297 distinct Python source contents, 516 archive-member source occurrences in those distinct archives, and 2,249 occurrences if repeated nested inclusion paths are counted. Archive reading and Python parsing reported no errors. The source-role audit identifies 130 nominal complete standalone checker sources; this classification is not a formal soundness certification of every historical program.

The comparison includes complete programs in `reference/` and `ancillary/` directories. In particular, the 672-byte `zero_product.py` must count: excluding it because of its directory name would overstate the improvement. Every one of the 23 nonchecker sources that fell below any coordinate of the baseline envelope was inspected; they were build, packaging, measurement, conversion, or test utilities. The complete census, role classifications, hashes, and per-archive minima are in `reference/audit/`.

Complete program	Tokens	AST	Norm. bytes	Gzip-9 raw
Constructed computation, explicit	277	317	684	403
Transient seed, structure	280	311	705	393
Counter congruence, zero-product	281	322	672	397
Flat computation	278	314	679	402
Old coordinate minima	277	311	672	–
New iterative checker	**276**	**305**	749	398
New byte profile	280	319	**652**	394

The token saving is one token (0.36%), the AST saving is six nodes (1.93%), and the normalized-byte saving is twenty bytes (2.98%). The first two belong to the iterative checker; the last belongs to the byte profile. *The family envelope is not the size of one program.* The iterative program has 746 actual source bytes; the byte profile has 652. The old raw-source minimum is 628 bytes, so the new byte profile does not improve that different metric. Normalized gzip sizes are 399 and 394 bytes, respectively. The separately supplied explanatory checker has 303 normalized tokens, 339 AST nodes, 1,401 normalized bytes, 2,051 raw bytes, and 897 raw gzip bytes. Its larger names and explanations are fully counted rather than hidden in a trusted dependency.

1.1 Measurement protocol

The unchanged inherited normalizer removes comments and genuine docstrings, parses the entire module, calls `ast.unparse`, appends one newline, and measures significant Python tokens, AST nodes, and UTF-8 bytes. The entire trusted program is counted, including the parser import, scope traversal, proof dispatcher, input, rejection, and acceptance output. Python and its standard JSON parser are shared infrastructure for the compared programs. The recorded runtime is CPython 3.12.14; changes to the Python AST or unparser can change metrics.

As requested, gzip is an *FYI measurement only.* We compress the exact delivered bytes at level 9, with timestamp zero and no stored filename. We also report compression of normalized bytes. This is reproduced by `gzip-9-n-c` for the measured size; the Python implementation uses the corresponding deterministic gzip settings. The zlib runtime is 1.3.2. No compression objective was used to select the mathematical axioms. A source can shrink while its gzip grows; the optional Boolean-tail loop experiment exhibited precisely that.

The current attained minima are upper bounds on the unknown best implementation size. They are not mathematical impossibility bounds. The operational target used here is the user's requested comparison against the best complete implementations in the actual supplied corpus. There is no claim that a globally simplest axiomatization has been found.

1.2 New and inherited contributions

The scalar iterator, paired conditional-closure binder, case/progress choice, flat computation packets, known-witness assumptions, and ordinary iPRA bridge are inherited. The present software contribution is an iterative continuation checker with persistent contexts and a smaller byte implementation using justified local refactorings. These are modest improvements to that presentation.

The main mathematical extension is the eventual-sequence model and full-arity universal-Horn obstruction in Section 5. The progressive-binder presentation in Section 6 supplies an alternative logical organization and a one-step universal-quantifier encoding. Neither is assigned the small checkers' measurements. Novelty here is relative to the inspected source corpus, not a literature-wide priority claim. The finite-parameter automorphism method itself already appears in the nested unary-arithmetic amalgam drafts [14]; a simpler product-model coverage counterexample is also inherited [15]. The new construction and its expanded scope, rather than those methods, are the distinct contribution asserted here. Both smaller obstruction appendices are explicitly attributed rederivations of current results.

2 The reference theory and the exact implemented presentation

2.1 Which iPRA is meant

Throughout, iPRA means *full first-order intuitionistic* primitive recursive arithmetic, not merely its equation consequences or its quantifier-free language. The signature contains a symbol for every fixed finite PR description, with its projection, composition, and primitive-recursion equations; zero and successor are included. The constructor axioms are successor injectivity and $Sx \neq 0$. Induction is allowed for every quantifier-free formula, in arbitrary retained contexts, with the usual fresh induction parameter. This convention accords with the distinction between first-order and quantifier-free presentations described in [13].

The underlying intuitionistic logical rules are hypothesis, weakening and reusable assumptions, implication introduction and elimination, conjunction introduction and its projections, disjunction introductions and case elimination, universal introduction and specialization, existential introduction and fresh-witness elimination, and falsity elimination. Equality has reflexivity and capture-free substitution. The usual eigenvariable restrictions apply. No law of excluded middle for arbitrary quantified formulas is added.

2.2 Numerical syntax and all numerical axioms

The implemented numerical grammar is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only the update h binds u; its other parameters stay fixed during the iteration. The two numerical axiom schemas, usable in any context, are exactly

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The update receives its preceding state, not the old counter. There is no numerical quantifier over functions or descriptions. Equivalently, introduce staged first-order symbols i_h for fixed earlier-stage bodies h. The description grammar is external finite syntax, and no uniformly total evaluator for all PR descriptions is postulated.

2.3 Logical syntax and all inference rules

Primitive object formulas can be taken to be

$$A ::= (s = t) \mid D_{x,y}A.$$

Contexts also contain internal clauses $Q_yB(y)$, whose intended meaning is $B(0) \to \forall yB(y)$. The executable format permits the conservative object abbreviation $Q_yB := D_{u,y}B$ with unused u. Thus native formulas with an explicit Q root are expanded to this abbreviation when comparing languages. Let Γ be a finite reusable context. Apart from I0 and IS, the following is the entire proof-building inventory:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma)\quad \mathsf{Hyp}, \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\quad \mathsf{Tr},$$

$$\frac{\Gamma \vdash Q_yB(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\quad \mathsf{QE},$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \quad \Gamma, Q_yB(a,y) \vdash A}{\Gamma \vdash A}\quad \mathsf{DE},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

The case choice $\varepsilon = 0$ is allowed for every matrix. The progress choice $\varepsilon = 1$ is allowed only when the matrix is a literal equation, not an equation below a binder. In DI, z is fresh for Γ, the witness t, and the fixed matrix parameters. In DE, a is fresh for Γ, the conclusion A, and the fixed matrix parameters. All displayed local assumptions are discharged; all substitutions avoid capture. In transport, F is a fixed finite formula template. These conditions are part of the rules and are checked by the program's opening and freshness mechanism.

Ordinary hypothesis use in the inherited clause-only presentation is derived using a constant clause and a fixed computation theorem. Conversely, the explicit Hyp rule above implements that derived rule. The displayed Q extension is conservative: DE followed by Hyp proves Q_yB from $D_{u,y}B$, while case DI and QE prove $D_{u,y}B$ from Q_yB. The unused coordinate is fresh, so neither conversion changes the matrix's parameters.

More explicitly, recursively erase every object occurrence of Q_yB to $D_{u,y}B$ with a fresh unused u, erasing the body in turn. An object-Q specialization is simulated by DE on this expanded D, followed by internal-clause QE on its base. Hypothesis, transport, and paired rules otherwise translate directly. If Hyp treats a retained internal Q clause as an object conclusion, case DI and that clause's QE supply the expanded D conclusion. Numerical substitution preserves literal-equation roots, so the progress restriction is preserved. This gives an explicit derivation translation, not merely two formulas with the same intended meaning.

Native falsity is $\bot_* := (0 = S0)$. There are no other logical or nonlogical axioms, reflexivity schema, numerical coverage axiom, ordinary-logic axioms, or induction rules in this primitive inventory. The next subsection explains their derivation.

2.4 Recovering all intuitionistic logic

The full inherited proof appears in Appendix A; the essential constructions are recorded here to make the scope explicit. Reflexivity follows by transporting the computation $I(0,t;u.0) = t$ along itself. Equality symmetry, transitivity, and congruence follow by the corresponding fixed motives.

For any equation E, progress DI with an unused witness gives $D_{u,y}E(y)$. DE and QE then combine its step with a supplied base proof, giving contextual equation induction. Case DI with an unused witness similarly gives numerical cases

$$\frac{\Gamma \vdash F(0) \quad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)},$$

where z is fresh for the retained context. This preliminary rule does not use disjunction or a hidden quantified induction instance.

Define the numerical selector $C(n,a,b) = I(n,a;u.b)$, with the update variable unused. It satisfies $C(0,a,b) = a$ and $C(Sn,a,b) = b$. For equations define

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \quad M_v(s=t, r=w) := (C(v,s,r) = C(v,t,w)).$$

Extend gates through paired binders and align unequal binder depths using the provable padding $A \dashv\vdash D_{u,y}G_yA$. This gives selectors for arbitrary formulas, with $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$. Let $p(z) = I(z,0;u.C(u,1,0))$; the finite computations $p(1) = 1$ and $p(2) = 0$ suffice. The logical macros are

$$\begin{aligned} A \Rightarrow_* B &= D_{u,z}M_z(A,B), \\ A \wedge_* B &= D_{u,z}G_zM_{p(z)}(A,B), \\ \exists_* xA(x) &= D_{x,y}G_yA(x), \\ A \vee_* B &= \exists_* xM_x(A,B), \\ \forall_* xA(x) &= A(0) \wedge_* D_{u,z}G_zA(z). \end{aligned}$$

Here u is unused and all binders are fresh. The implication and existential rules use case DI, DE, and QE. Conjunction projections specialize at 1 and 2. Disjunction elimination uses numerical cases on its selector. These rules derive $(x=0) \vee_* \exists_* y(x = Sy)$; that coverage formula permits universal specialization even when the target occurs in the retained context. Under $0 = 1$, the selector gives $s = C(0,s,t) = C(1,s,t) = t$. External induction on formula syntax therefore gives explosion. No quantified excluded middle is used.

2.5 Bi-interpretability, rather than just agreement on numerals

Let T_I be ordinary intuitionistic first-order equality logic in the scalar signature, with I0/IS, contextual literal-equation induction, constructor coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, and the boundary axiom $(0 = 1) \to \bot$. These are axioms of the ordinary realization; their native translations have been derived above and are not extra axioms of the small kernel.

Interpret the native binder and internal clause in T_I by

$$(D_{x,y}B)^\circ = \exists x(B^\circ(x,0) \to \forall y B^\circ(x,y)), \qquad (Q_yB)^\circ = B^\circ(0) \to \forall y B^\circ(y).$$

Keep the entire numerical domain and actual equality. Case DI is sound by coverage, progress DI by contextual equation induction, and the other rules by their displayed ordinary meanings. Conversely the macros derive all ordinary rules, coverage, and equation induction, as just described.

The scalar arithmetic bridge is substantial and is retained in full. Appendix B constructs a finite scalar term P and proves $P(0) = 0$, $P(Sn) = n$ internally. It uses only equation induction and numerical cases, with passive parameters fixed. Appendix C constructs a faithful equality characteristic, proves its two-value Boolean range, and translates every quantifier-free formula A to a Boolean equation $b_A = 0$. Equation induction then gives all quantifier-free induction. Appendix D constructs pairing and compiles each fixed counter-aware primitive recursion into a scalar iteration.

For each numerical composite, external induction on its finite description reduces the comparison to two functions satisfying the same recursion equations. Internal equation induction proves their equality at arbitrary counters and parameters. Thus the composite isomorphisms have graph $x = y$ over the whole domain, not merely over standard numerals or a chosen cut.

The essential logical composite is

$$D_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y B(x,y)).$$

Forwards, DE supplies the clause under a fresh witness; under its base assumption QE supplies each instance, so the ordinary macros form the conditional universal and existential. Backwards, eliminate the existential and use case DI, obtaining its successor premise from the supplied conditional and universal specialization. The Q composite is the unused-witness special case. External induction on formulas gives both complete logical round trips.

Theorem 2.1 (Exact scope of the implemented theory)**.** *The displayed native theory and full first-order iPRA are whole-domain, actual-equality bi-interpretable through the stated ordinary realization and logical macros. Both numerical composite isomorphisms are internally $x = y$. Effective finite proof translations preserve designated contradiction, giving equiconsistency.*

Proof. The logical reconstruction and both binder composites are proved in Appendix A. The internal numerical compiler and uniqueness arguments are supplied by the subsequent three appendices. The explicit separation bridge in the ordinary theory identifies native $0 = 1$ with ordinary falsity. Together these prove all claims. This is an external metatheorem, not an assertion that either theory proves its own consistency. □

3 What the checker changes actually do

3.1 Persistent contexts and iterative continuations

The old flat checker has a recursive call for a major premise and another recursive call for the continuation after each instruction. The new `checker_loop.py` keeps recursion for major premises but executes the continuations in a `while` loop. Its context is an immutable tuple. Appending a local assumption with tuple `+=` creates a new tuple bound only in the current invocation. A recursively checked major receives the preceding tuple and cannot alter the caller's assumptions.

This persistence condition is mathematical, not just a coding preference. Consider $F = (0 = 1)$ and the true vacuous closure $M = D_{x,y}F$. Case introduction proves M while locally assuming F. A checker that appends into a shared mutable list can leak that local F back into the caller and then accept falsity. The new regression `tests/new/context_isolation.json` exhibits exactly this error in an intentionally mutated control. The genuine tuple checker rejects it.

Lemma 3.1 (Continuation simulation)**.** *On canonical finite certificates, and with unbounded execution resources, the iterative checker has the same successful derivations as the original flat-computation checker, with the same fresh-name allocation order.*

Proof. Maintain the invariant that the loop's current goal, context contents, global freshness floor, and unconsumed instruction stack equal those at the corresponding old recursive call. Hypothesis closure is the same membership test. The major-premise recursive call uses the same preceding context, and any global fresh-name allocations it performs persist in both programs. Every branch constructs the same next goal and added assumption. Tuple concatenation has the same contents as old list concatenation and does not change any earlier context. The subsequent loop iteration therefore establishes the invariant again. Goal opening and assumption opening have no side effects on the fresh-name floor, so interchanging their order does not alter a successful transition. Induction on the finite checking trace proves the claim in both directions. □

The iterative version removes recursion-depth dependence on the number of continuation steps. It does not remove recursion limits on nested major proofs, syntax traversal, or JSON parsing, and it does not make a claim of lower asymptotic memory use for contexts.

3.2 Why the byte refactorings are sound

The compact byte profile combines the same flat interface with ordinary source refactorings, including the corpus's zero-product list test, a short-circuit form of guarded elimination, a Boolean form of the fresh-image choice, and direct input initialization. These are software changes; the twenty-byte saving is not attributed to a new arithmetic basis.

For JSON values, `a*0==[]` recognizes arrays in the opener: lists give the empty list, numerical values give numerical zero, and strings give an empty string. Unsupported values either fail there or in the numerical comparison, as in the original validator. No user-provided text is executed as Python.

At a formula root, successful opening preserves a list's outer arity; for a scalar, its returned root is independent of the supplied numerical image. Hence if a guarded elimination has matched its target $a = i(t, u)$ and its source $i(t, s)$ is falsy, that source equals the same falsy target. Replacing an explicit conditional by `conditionandi(t,s)ora` therefore preserves the old result, including that boundary case. It does not discard a failed major: the major is checked first.

Likewise `(morlen(a)%2==m)andk` selects the same fresh name or numerical zero as the original conditional; its false value is Python `False`, which the decoder treats as zero. The driver still prints `ACCEPT` with status zero on success and `REJECT` with nonzero status on failure. The entire driver remains inside the measured module.

3.3 Syntax, freshness, and arithmetic packets

The raw packet format is inherited and fully documented in `code/INHERITED_FORMAT.txt`. Input is a JSON array ending with the requested closed formula. Earlier four-field packets $[m, s, u, t]$ are consumed from the end. An unused prefix is inert. The same list grammar has two sorted interpretations:

Arity	Formula position	Numerical position	Depth increment
0	truth	zero	no children
1	conditional closure Q	successor	2
2	equation	annotated zero	1
3	paired binder D	scalar iterator	4

Canonical coordinates are integers. Negative names designate allocated rigid parameters; positive coordinates are local binder slots. The global lower bound begins at zero and decreases at each continuation. A leaf is retained only in the active scope interval; opening replaces exposed numerical coordinates by a checked image. The same traversal checks the initial goal before allocating its first new name. The exact decoder includes checked dummy slots; these are syntax conventions, not additional arithmetic operations.

A truthy m is first proved. Matching Eq/Q endpoint packets perform transport or specialization; unmatched ones keep the goal and add only truth. A ternary major opens the DE clause. A `None` major selects arithmetic and exposes the three fields s, u, t as counter, initializer, and update. A zero counter generates I0. A successor counter is unpacked eagerly using `list(*s)`; this prevents malformed counter shapes from slipping through when the update ignores its state. Other falsy major markers select the two permitted DI choices. The common continuation then opens the goal and the one new assumption with the correct fresh images.

The arithmetic equations produced by flat packets include the previously proved known-zero-instance administrative convention. The generated equation's zero completion is an I0 or IS instance; the old goal and context exclude the new name. Substitution of zero and ordinary cut therefore justify its continuation. For canonical packets this reduces to insertion of the familiar computation equation. The new code does not change that generator.

The iterative compact opener uses the existing exception-based iterable variant. Canonical array/integer proofs terminate structurally; some additional empty-container aliases decode as zero or truth, while nonempty strings can require the Python recursion limit to reject. This is an inherited raw-input limitation, not a new numerical principle or an ideal total-parser claim. The byte profile retains an explicit list test. The claimed canonical proof translations do not require those noncanonical aliases.

4 Verification, files, and the remaining target

The delivered reproduction command is

```
python3 reproduce.py
```

It verifies the file manifest, remeasures the complete checkers and six frozen comparison programs, checks the inherited proof corpus under ordinary and optimized Python, and runs the targeted context isolation and deep-continuation controls. Its output is a receipt, not a formal proof of universal soundness.

All 31 inherited cases cover actual contextual equation induction, nonvacuous fresh-witness elimination, transport below a closure, arithmetic sources, malformed counters, unallocated names, wrong endpoints, and forbidden compound progress. The new depth test contains 1,600 harmless arithmetic continuations followed by a valid computation. At an explicitly fixed recursion limit of 1,000, the loop and its readable version accept, while the old recursive checker and the byte profile reject with `RecursionError`. This demonstrates a practical benefit without changing the unbounded-resource consequence relation.

The context-isolation control is deliberately stronger than checking that some normal proofs still pass: deleting the persistence property makes a false formula pass. The mutation is generated only in a temporary test directory; no unsafe checker is distributed as a candidate implementation.

The scalar predecessor construction, general PR compiler, and interpretation metatheorems are written proofs, not complete native certificates or Lean/Rocq developments. Finite tests do not

certify all models or all proof traces. The full Python interpreter and standard JSON library are not formally verified. These are the material limits of the evidence.

What remains unachieved. No single delivered checker strictly improves 277 tokens, 311 AST nodes, and 672 normalized bytes simultaneously. The progressive binder has a complete mathematical treatment but no smaller implemented checker. Neither the finite-parameter obstruction nor the two smaller obstruction lemmas prove a source-size lower bound for every possible language, domain, or presentation of iPRA. The requested significant positive breakthrough remains open in this investigation. The new obstruction does, however, rule out a substantial class of algebraic shortcuts without relying on same-domain term representability alone.

5 A finite-parameter obstruction to removing numerical cases

Antecedents and precise contribution. The finite-parameter involution versus definable-linear-order method, including the treatment of quotient representatives, is already explicit in the nested unary-arithmetic amalgam drafts [14]. The simpler product-model independence of coverage is also already present [15]. The construction below extends this line of argument using a countable eventually constant power for full-arity arithmetic, all true universal Horn laws, and arbitrary finitary expansions with atomic or conjunctive induction. No priority for the automorphism method or the elementary product observation is claimed.

This section proves a stronger limitation than failure of one proposed term compiler. A countable model built from finitary objects obstructs *any* ordinary definable bi-interpretation with iPRA, even when finite tuples of parameters, different interpretation domains, finite-dimensional coding, and definable quotients are allowed. It applies to the ordinary first-order realization of the numerical theory left after removing constructor coverage. The scope concerning native calculi with unusual logical connectives is stated explicitly below.

5.1 The ordinary theory under consideration

Use the usual one-sorted first-order signature with a function symbol for every fixed primitive-recursive description. Different descriptions may name the same standard function. Let T_{alg} have ordinary intuitionistic first-order logic with equality, all PR defining equations, constructor separation, and induction for each literal equation $E(n, \vec{p})$:

$$\forall \vec{p}\,[(E(0,\vec{p}) \wedge \forall n\,(E(n,\vec{p}) \to E(Sn,\vec{p}))) \to \forall n\,E(n,\vec{p})]\,. \tag{EI}$$

One may take separation to include both $0 \neq Sx$ and successor injectivity; the construction below satisfies them. Equivalently, the weaker boundary axiom $0 \neq 1$ may be used when the others are derived from the chosen PR descriptions. Constructor coverage is deliberately absent.

Although the theory is intuitionistic, it has ordinary classical structures as one-world models. Every interpretation or definable isomorphism proved intuitionistically must remain valid in these structures. They therefore suffice for the obstruction.

5.2 A countable model of all literal-equation induction

Let $\mathcal{E}$ have domain

$$\{a : \mathbb{N} \to \mathbb{N} : \text{there exist } c \in \mathbb{N} \text{ and finite } F \subseteq \mathbb{N} \text{ with } a(i) = c \text{ for all } i \notin F\}\,. \tag{Domain}$$

These are the eventually constant sequences. Each element has a finite description consisting of its eventual value and its finitely many exceptional coordinate values, so the domain is countable. Interpret every PR symbol pointwise using its ordinary standard numerical meaning. Constants are constant sequences; in particular

$$0(i) = 0, \qquad S(a)(i) = a(i) + 1.$$

Closure and arithmetic equations. If $a_1, \ldots, a_r$ are eventually constant, then outside the union of their finite exception sets every tuple of their coordinate values is the same. Applying any fixed r-ary PR function consequently gives another eventually constant sequence. Every defining equation is true coordinatewise. Successor is injective and its range omits zero, also coordinatewise. Equality in this structure is actual equality of sequences.

Lemma: every instance of EI holds in $\mathcal{E}$. Fix an equation $E(n, \vec{p})$ and an arbitrary finite tuple of parameter sequences $\vec{p}$. For each coordinate i, put

$$U_i = \{m \in \mathbb{N} : E(m, \vec{p}(i)) \text{ holds in the standard natural numbers}\}.$$

Term evaluation is pointwise, so

$$\mathcal{E} \models E(a, \vec{p}) \quad \Longleftrightarrow \quad (\forall i)\ a(i) \in U_i. \qquad \text{(Coordinate factorization)}$$

Suppose the antecedent of EI holds. Its base gives $0 \in U_i$ for every i. Fix i and $m \in U_i$, and form the finite-support sequence

$$a_{i,m}(j) = \begin{cases} m, & j = i, \\ 0, & j \neq i. \end{cases}$$

This element belongs to $\mathcal{E}$ and satisfies $E(a_{i,m}, \vec{p})$: at i this is the choice of m, and at every other coordinate it is the known base case. The global induction-step premise gives $E(Sa_{i,m}, \vec{p})$, hence $m + 1 \in U_i$. Thus each U_i contains zero and is successor closed. External ordinary induction in the standard natural numbers gives $U_i = \mathbb{N}$ for every i. Coordinate factorization now gives $E(a, \vec{p})$ for every $a \in \mathcal{E}$. This proves the instance and hence the lemma. □

This argument applies to every fixed PR term equation, with arbitrary finite parameter tuples. It proves the full axiom schema and therefore also validates the contextual equation-induction rule under arbitrary retained assumptions. It is not merely a verification of induction for parameter-free equations.

What fails. Let e_i be the sequence that is one at i and zero elsewhere. It is neither zero nor a successor, since every successor sequence has a positive value at every coordinate. Therefore constructor coverage fails at e_i.

There is also a direct failed quantifier-free induction instance. Let sg be the usual PR sign function, zero at zero and one at successors, and let

$$\varphi(n) := (\mathrm{sg}(n) = 0) \vee (\mathrm{sg}(n) = 1).$$

Its base holds, and $\varphi(Sa)$ holds for every a, since $\mathrm{sg}(Sa)$ is the constant-one sequence. But $\mathrm{sg}(e_i) = e_i$ is neither constant zero nor constant one, so $\varphi(e_i)$ fails. The missing feature is a disjunctive range assertion, not an inability to perform numerical computation.

For precision, the standard PR equality characteristic still reflects actual equality in this model: its value is the zero sequence exactly when its two arguments are equal at every coordinate. What fails is the assertion that this characteristic is always *globally* equal to zero or globally equal to one. These are different claims.

5.3 Finite parameters cannot remove the symmetry

Lemma. For every finite tuple $\vec{a}$ of elements of $\mathcal{E}$, the parameter expansion $(\mathcal{E}, \vec{a})$ has a nonidentity automorphism of order two.

Proof. Let F be the union of finite exception sets for all entries of $\vec{a}$. Choose distinct coordinates $i, j \notin F$. Exchange those two coordinates in every sequence and leave all others fixed. This map preserves eventual constancy and commutes with every pointwise PR operation. It fixes each entry of

$\vec{a}$, because that entry takes its eventual value at both i and j. It is nonidentity because it interchanges e_i and e_j, and applying it twice is the identity. □

In fact the same construction gives every finite permutation of coordinates outside F, but an order-two automorphism is enough. It follows already that no strict linear order on $\mathcal{E}$ can be defined with a finite tuple of parameters.

5.4 Why bi-interpretability cannot conceal this symmetry

Every ordinary model of iPRA has a parameter-free definable strict linear order, for example

$$x < y \quad \Longleftrightarrow \quad (Sx) \dot{-} y = 0,$$

where $\dot{-}$ is truncated subtraction. The usual order laws are elementary PRA theorems, proved with quantifier-free induction in the language of PR comparisons. Thus every automorphism of an iPRA model preserves a strict linear order.

An order-preserving automorphism σ of finite order is the identity. If $a < \sigma(a)$, repeated application gives a strict cycle $a < \sigma(a) < \cdots < \sigma^r(a) = a$. If $\sigma(a) < a$, the same argument with reversed inequalities gives a contradiction. Linearity therefore gives $\sigma(a) = a$ for every a. Consequently the automorphism group of every iPRA model has no nonidentity element of finite order.

Here is the needed consequence of bi-interpretability, with its proof. Suppose a structure M interprets N by a finite-dimensional definable domain and a definable equality relation, possibly a quotient. An automorphism of M fixing the finitely many interpretation parameters acts coordinatewise on representing tuples and therefore induces an automorphism of N. This is a group homomorphism.

Suppose, in addition, N interprets a copy of M and the composite copy is definably isomorphic to M, as required for a bi-interpretation. Collect into one finite tuple $\vec{c}$ all parameters used by the first interpretation, the definable composite isomorphism, and finite representatives in M of parameters used by the second interpretation. The induced homomorphism

$$\mathrm{Aut}(M/\vec{c}) \longrightarrow \mathrm{Aut}(N) \qquad \text{(Automorphism action)}$$

is injective. Indeed, if σ induces the identity on N, it induces the identity on the interpreted copy of M. The graph of the composite isomorphism is definable over parameters fixed by σ, so this isomorphism commutes with the two automorphism actions. Its injectivity then forces σ to be the identity on M. Only this one composite is needed for the argument.

This proof accommodates finite tuples, definable quotients, different interpretation domains, and finite parameter data. It does not assume that either interpretation has the identity domain or preserves each element literally.

Theorem. The structure $\mathcal{E}$ is not bi-interpretable with any ordinary model of iPRA by ordinary definable interpretations with finite parameter data. Consequently T_{alg} and iPRA are not bi-interpretable in that sense, even with finite parameters.

Proof. For a hypothetical bi-interpretation, take the finite tuple $\vec{c}$ in Automorphism action. The preceding symmetry lemma supplies a nonidentity element of order two in $\mathrm{Aut}(\mathcal{E}/\vec{c})$. Its injective image would be a nonidentity finite-order automorphism of an iPRA model, which the definable linear order excludes. Since $\mathcal{E}$ is a model of T_{alg}, any theory-level bi-interpretation would apply to it and give the same contradiction. □

5.5 All true universal Horn laws still do not suffice

The obstruction persists after adjoining *any collection* of universal Horn sentences in the fixed PR signature that are true in the standard natural numbers. Here a universal Horn sentence has the form

$$\forall \vec{x}\,(E_1(\vec{x}) \wedge \cdots \wedge E_r(\vec{x}) \to F(\vec{x})),$$

where the premises are equations and the conclusion is an equation or falsity; the list of premises may be empty.

To verify preservation directly, take any tuple of eventually constant sequences satisfying all premises. At each coordinate, the numerical tuple satisfies all corresponding standard premises. Truth of the Horn law in $\mathbb{N}$ gives its equation conclusion at that coordinate, and doing this at every coordinate gives actual equality of the output sequences. If the conclusion is falsity, even one coordinate contradicts the assumed standard truth of the Horn law, so its premises cannot all hold. No closure-under-products theorem from model theory is needed for this verification.

Thus $\mathcal{E}$ satisfies not just the defining equations but every true universal Horn arithmetic law, while still satisfying EI and still having the same finite-parameter symmetries. Even the theory of all such true Horn laws together with literal-equation induction cannot be bi-interpretable with iPRA in the stated sense.

Within this fixed numerical signature, adding more true equations or universal Horn arithmetic axioms therefore cannot repair the omission of cases. An additional ingredient must exclude the model; constructor coverage and Boolean-range disjunctions are concrete examples that do so. This is not a prohibition on arbitrary alternative domains, new operations with different intended meanings, or other non-Horn axiomatizations.

Generalization to arbitrary finitary expansions. The same argument applies to any ordinary finitary first-order expansion $\mathbb{N}^*$ of the standard structure $(\mathbb{N}, 0, S)$, not only to the PR signature. Interpret each additional function pointwise on eventually constant sequences, and each relation by requiring its standard interpretation at every coordinate. Closure under the functions follows from eventual constancy; atomic formulas factor coordinatewise by definition. The finite-support-sequence proof therefore validates induction for any atomic formula, and even for any finite conjunction of atomic formulas. All universal Horn sentences true in $\mathbb{N}^*$ remain true by the same coordinate argument. Coordinate transpositions preserve all these functions and relations and still fix any prescribed finite tuple of parameters. Consequently such Horn laws plus atomic, or finite-conjunction, induction cannot force a theory bi-interpretable with iPRA by ordinary definable interpretations with finite parameter data.

This generalization still starts from an intended standard zero and successor on $\mathbb{N}$ and uses ordinary model-theoretic interpretations. It is not a theorem about every exotic semantics or every possible induction mechanism on another kind of object.

5.6 Exact application to the progressive native calculus

Interpret the native progressive symbols by

$$\mathsf{P}_y B = \forall y(B(y) \to B(Sy)), \qquad \mathsf{H}_{x,y} B = \exists x\, \forall y(B(x,y) \to B(x,Sy)).$$

Every progressive-calculus rule other than BoolCases (equivalently, the derived constructor Cases interface) is sound in T_{alg}: transport, HI, HE, and Step have their ordinary logical meanings, and

equation-only Iter is EI. Consequently $\mathcal{E}$ is also a sound standard-semantics model of that case-free rule system. The theorem rules out rescuing its ordinary first-order realization by a finite-parameter definable bi-interpretation on a different domain or quotient.

There is a necessary distinction. Soundness of a native proof calculus in an ordinary theory is not by itself a proof that the native calculus derives all the ordinary logical rules of that theory. Nor does an arbitrary proof translation that changes the meanings of logical connectives automatically give an ordinary model interpretation. The automorphism theorem applies to standard model-theoretic bi-interpretability, or to a native proposal whose claimed interpretations are proved to induce those definable model interpretations under the displayed semantics. It is not presented as a theorem about every conceivable nonstandard meaning of "proof equivalence."

The result is also not a lower bound on Python source size or on every possible axiomatization of iPRA. It establishes a precise obstruction to a substantial class of proposed arithmetic simplifications, including all purely equational or true universal Horn repairs in this signature. It does not exclude equiconsistency, nor does the automorphism argument by itself exclude interpretations lacking a definable recovery of the source model. The bi-interpretation composite is the essential extra requirement used by the proof.

6 A progressive binder presentation of iPRA

This section gives a different, complete presentation of iPRA in which every binder introduction has the same progress form. The induction restriction moves to elimination. Boolean case analysis must be retained explicitly; deleting it is not justified here. The result is a mathematical alternative, not a measured improvement over the main checker.

6.1 Numerical syntax and complete rule inventory

Numerical terms are variables, 0, St, and $R(n, a; u, v.h)$. Only the fixed finite update body h binds u, v; other parameters are held fixed. The numerical computation schemas are

$$R(0, a; u, v.h) = a, \qquad R(Sn, a; u, v.h) = h[n/u, R(n, a; u, v.h)/v]. \tag{PR}$$

All substitutions avoid capture, and the successor substitution is simultaneous. Equivalently use a staged first-order symbol for each fixed body. This is ordinary counter-aware primitive recursion, so zero, successor, projections, composition, and arbitrary fixed PR descriptions compile directly. There is no uniform evaluator over quantified descriptions.

Object formulas have grammar

$$A ::= (s = t) \mid \mathsf{H}_{x,y} A.$$

Internal clauses are $\mathsf{P}_y A(y)$, where A is an object formula. Contexts are reusable finite lists of objects and clauses. An object or clause is available when it is a context member; a proved object is also available. Clauses are not object goals. Put $\bot_* = (0 = S0)$.

Besides the computation schemas, the complete primitive rule inventory is

$$\frac{\Gamma \rhd s = t \quad \Gamma \vdash A(s)}{\Gamma \vdash A(t)} \ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, z) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{H}_{x,y} B(x, y)} \ \mathsf{HI},$$

$$\frac{\Gamma \rhd \mathsf{H}_{x,y} B(x, y) \quad \Gamma, \mathsf{P}_y B(a, y) \vdash A}{\Gamma \vdash A} \ \mathsf{HE},$$

$$\frac{\Gamma \rhd \mathsf{P}_y B(y) \quad \Gamma \vdash B(t)}{\Gamma \vdash B(St)} \ \mathsf{Step},$$

$$\frac{\Gamma \rhd \mathsf{P}_y E(y) \quad \Gamma \vdash E(0)}{\Gamma \vdash E(t)} \ \mathsf{Iter}, \qquad E \text{ a literal equation},$$

and Boolean cases

$$\frac{\Gamma, \mathrm{sg}(t) = 0 \vdash A \quad \Gamma, \mathrm{sg}(t) = 1 \vdash A}{\Gamma \vdash A} \ \mathsf{BoolCases}.$$

Here $1 = S0$ and $\mathrm{sg}(n) = R(n, 0; u, v.1)$, so $\mathrm{sg}(0) = 0$ and $\mathrm{sg}(Sn) = 1$. In HI, z is fresh for Γ, t and the passive parameters. In HE, a is fresh for Γ, A and the passive parameters. BoolCases has no eigenvariable. The displayed local assumptions and clauses are discharged. There is no primitive hypothesis-conclusion rule: context membership supplies availability only, and object identity is recovered by the expansion below. There are no other logical axioms or induction rules.

The intended ordinary meanings are

$$(\mathsf{P}_y B)^\circ = \forall y (B^\circ(y) \to B^\circ(Sy)),$$

$$(\mathsf{H}_{x,y}B)^\circ = \exists x\,\forall y\big(B^\circ(x,y) \to B^\circ(x,Sy)\big). \qquad \text{(Meaning)}$$

HI is unrestricted logical introduction. Iter is the only use of induction, and it is explicitly restricted to literal equations.

6.2 Elementary derived operations

Structural weakening, contraction, exchange, substitution, renaming, and assumption cut are admissible by induction on derivations, with freshening of eigenparameters before substitution. The equation $R(0,t;u,v.0) = t$ and transport derive $t = t$, followed by symmetry, transitivity, and numerical congruence.

An equation assumption $s = t$ transports reflexivity $s = s$ to $s = t$. For a paired assumption $\mathsf{H}B$, HE supplies a fresh witness a and $\mathsf{P}B(a,-)$. Introduce the same paired object at a; the induction hypothesis on formula depth recovers its assumed matrix $B(a,z)$, and Step yields $B(a,Sz)$. Thus every object identity has a finite expansion.

Literal-equation induction is derivable. Given $E(0)$ and a derivation of $E(Sz)$ from $E(z)$, HI with an unused witness gives $\mathsf{H}_{x,y}E(y)$. HE supplies its progress clause, and Iter gives $E(t)$. Conversely Iter is sound in ordinary iPRA by equation induction under the retained assumption $E(0)$.

Boolean cases recover constructor cases. Put $P(n) = R(n,0;u,v.u)$ and $C(n,a,b) = R(n,a;u,v.b)$. Their equations give $P0 = 0$, $P(Sn) = n$ and the usual selector endpoints. Literal-equation induction proves

$$C(\mathrm{sg}(x),0,x) = x, \qquad C(\mathrm{sg}(x),x,S(Px)) = x. \qquad \text{(Boolean coverage identities)}$$

Each base is a direct zero computation, and each successor step is a direct successor computation that does not use its induction hypothesis. Consequently $\mathrm{sg}(t) = 0$ gives $t = 0$ by the first identity, and $\mathrm{sg}(t) = 1$ gives $t = S(Pt)$ by the second. It follows that the constructor rule

$$\frac{\Gamma, t = 0 \vdash A \quad \Gamma, t = Sz \vdash A}{\Gamma \vdash A}\ \mathsf{Cases}$$

is admissible, with z fresh for Γ, t, A: use BoolCases, and in its second branch substitute $P(t)$ for the fresh z in the supplied successor derivation. Conversely constructor Cases proves BoolCases by the sign equations. Thus the two case interfaces have exactly the same consequences. The missing principle is isolated to the two-value range of sign; equation induction already supplies the needed numerical identities. In the rest of the proof, Cases denotes this derived constructor rule.

Binder congruence. Suppose $A(x,y)$ and $B(x,y)$ are uniformly interderivable with x, y absent from the retained context. From $\mathsf{H}A$, HE gives the progress clause of $A(a,y)$. To introduce $\mathsf{H}B$ at the same witness, convert the local $B(a,z)$ to $A(a,z)$, apply Step, and convert $A(a,Sz)$ to $B(a,Sz)$. Reverse the conversions for the opposite direction. Hence H preserves uniform equivalence. This argument needs neither Cases nor compound induction.

6.3 Gates and selectors on arbitrary formulas

Let $C(n,a,b)$ be the fixed PR numerical selector with $C(0,a,b) = a$ and $C(Sn,a,b) = b$. It is directly represented by $R(n,a;u,v.b)$. Define a numerical gate on formulas by

$$G_v(s = t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{H}_{x,y}A) := \mathsf{H}_{x,y}G_vA,$$

freshening bound variables first. External formula induction, numerical computation, and binder congruence give

$$\vdash G_0 A, \qquad G_{Sv} A \dashv\vdash A.$$

For the first assertion at a binder, HI ignores its local assumption and uses the induction hypothesis for the successor matrix.

Padding is again available:

$$A \dashv\vdash \mathsf{H}_{u,y} G_y A \qquad (u, y \text{ fresh for } A). \tag{Padding-H}$$

Forwards HI ignores its local assumption and uses the positive-gate equivalence. Backwards HE supplies a progress clause; $G_0 A$ is a theorem, so one Step gives $G_1 A$ and hence A.

The selector on formulas is defined exactly by finite structural recursion:

$$M_v(s = t, r = w) := (C(v, s, r) = C(v, t, w)),$$

$$\begin{aligned} M_v(\mathsf{H}_{x,y} A, \mathsf{H}_{x,y} B) &:= \mathsf{H}_{x,y} M_v(A, B), \\ M_v(E, \mathsf{H}_{x,y} B) &:= \mathsf{H}_{x,y} M_v(G_y E, B), \\ M_v(\mathsf{H}_{x,y} A, E) &:= \mathsf{H}_{x,y} M_v(A, G_y E), \end{aligned}$$

where E is an equation. Bindings are aligned by fresh renaming. The maximum binder depth decreases in every recursive call. Computation, padding, and congruence give

$$M_0(A, B) \dashv\vdash A, \qquad M_{Sv}(A, B) \dashv\vdash B. \tag{Selector-H}$$

Cases therefore permits choosing either endpoint at an arbitrary control, including when the retained context mentions that control. In particular proofs of both A and B give a proof of $M_t(A, B)$.

6.4 All intuitionistic logical connectives

Use $\mathsf{T} = (0 = 0)$. Choose fixed PR functions p, q, d with

$$p(0) = 0, \quad p(Sn) = C(p(n), 1, 0),$$

$$q(0) = 0, \quad q(Sn) = C(p(n), q(n), S(q(n))),$$

$$d(0) = 0, \quad d(Sn) = SS(d(n)).$$

They are parity, integer half, and doubling in the standard model. The following equations are proved internally by literal-equation induction and their defining computations:

$$p(d(n)) = 0, \quad p(Sd(n)) = 1, \quad q(d(n)) = n, \quad q(Sd(n)) = n. \tag{Clock}$$

First induct on the single equation $p(d(n)) = 0$. At a successor, two parity updates send zero to one and back to zero. The identity $p(Sd(n)) = 1$ then follows by one computation. Next induct on the single equation $q(d(n)) = n$. The two half updates at $d(n)$ and $Sd(n)$ first leave the value unchanged and then increment it, using the already proved parity equations. Finally $q(Sd(n)) = n$ follows by its defining zero-parity update. Thus the argument uses two single-equation inductions in sequence, not conjunction induction or induction on any compound formula.

Define the logical macros

$$\begin{aligned} A \Rightarrow_H B &:= \mathsf{H}_{u,z} M_z(A, B), \\ \exists_H x\, A(x) &:= \mathsf{H}_{x,z} G_z A(x), \\ A \wedge_H B &:= \mathsf{H}_{u,z} G_z M_{p(z)}(A, B), \\ A \vee_H B &:= \exists_H x\, M_x(A, B), \\ \forall_H x\, A(x) &:= \mathsf{H}_{u,z} M_{p(z)}(\mathsf{T}, A(q(z))), \\ \bot_H &:= (0 = S0). \end{aligned} \tag{H-macros}$$

All displayed unused coordinates and binders are fresh. The universal macro uses alternating true positions: its matrix is true at $d(n)$ and equivalent to $A(n)$ at $Sd(n)$. These macros are effective translations of fixed finite syntax, not numerical truth predicates or internal quantification over formula codes.

Implication. Given $\Gamma, A \vdash B$, apply HI to the implication macro. Its local assumption is $M_z(A, B)$. Cases splits $z = 0$ and $z = Sw$. The first branch provides A and then B; the second already provides B. In both branches the successor matrix is B. For elimination, HE supplies the progress clause; A is its zero matrix, and Step at zero gives its first positive matrix B.

Existential quantification. From $A(t)$ use HI at witness t, ignoring the local gate assumption. For elimination, HE gives a fresh witness a and its progress clause. The zero gate is a theorem; Step at zero gives $A(a)$. The ordinary fresh-witness elimination convention follows directly from HE.

Conjunction. If both A and B are proved, Cases and the selector endpoints prove every successor matrix, so HI applies. Conversely HE gives the progress clause. Its zero gate is a theorem, so Step at zero gives the matrix at 1, namely B. Step once more gives the matrix at 2, namely A, using $p(1) = 1$ and $p(2) = 0$.

Disjunction. The two introductions choose existential witnesses zero and one. For elimination, use existential elimination at a fresh witness a; Cases splits its zero and successor alternatives. The selector endpoints then give the respective supplied branch assumptions. The final conclusion is arbitrary, with the usual retained context.

Universal quantification. Suppose $\Gamma \vdash A(x)$ with x fresh for Γ. After numerical substitution, both T and $A(q(Sz))$ are proved. Cases and the selector endpoints therefore prove the successor matrix of the universal macro; HI may ignore its local assumption. For specialization, HE provides the progress clause. The matrix at $d(t)$ is a theorem by Clock and its true endpoint. Step at $d(t)$ gives the matrix at $Sd(t)$, which Clock identifies with $A(t)$. This proof does not iterate a compound progress clause.

Falsity and coverage. Under $0 = 1$, numerical computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for all numerical s, t. External formula induction then gives every object: for an H node, HI ignores the local assumption and uses the induction hypothesis on its successor matrix. Thus explosion is derived. Cases also proves the ordinary coverage macro $(n = 0) \vee_H \exists_H v\, (n = Sv)$ by its two evident introductions.

Consequently every ordinary intuitionistic natural-deduction rule is derived, including all eigenvariable restrictions. No excluded middle for arbitrary quantified formulas has been used.

6.5 Whole-domain bi-interpretability

For a transparent comparison, let T_H be ordinary intuitionistic first-order logic with equality in the staged PR signature, with the displayed PR computation schemas, literal-equation induction, constructor coverage, and $(0 = 1) \to \bot$. The native rules translate to T_H under Meaning: HI, HE, and Step are ordinary logical rules; Iter is equation induction; BoolCases is the sign range consequence of coverage. Conversely the native system derives all the ordinary logical rules, coverage, and equation induction, with separation translating to the identity implication $(0 = 1) \Rightarrow_H (0 = 1)$.

The usual actual-equality characteristic construction for PR computation plus coverage, separation, and literal-equation induction proves every quantifier-free induction instance. In particular this is the same arithmetic completion theorem proved in the arithmetic appendix of the main report. Its hypotheses have all been established here. Thus T_H and iPRA have the same consequences in their shared staged PR language; the domain is the entire numerical domain and equality is actual equality.

It remains to check the native formula composite, not merely the logical macros. For every fixed native matrix B one has

$$\mathsf{H}_{x,y} B(x, y) \dashv\vdash \exists_H x \, \forall_H y \, (B(x, y) \Rightarrow_H B(x, Sy)). \tag{H-composite}$$

Forwards, HE supplies a fresh a and its progress clause. At fresh y, assume $B(a, y)$ and use Step to obtain $B(a, Sy)$. Introduce implication and universal macros, then introduce the existential at a, and close HE. Backwards, eliminate the existential at fresh a. Specialize its universal at the fresh HI eigenparameter z. Under HI's assumption $B(a, z)$, implication elimination gives $B(a, Sz)$. HI at witness a and existential elimination finish. The analogous clause composite is the universal progress formula.

In ordinary T_H, the gate, selector, and Clock equations prove the intended meanings of every macro. The universal macro is worth emphasizing: its even-to-odd progress implications give every instance $A(n)$, while all instances of A make every progress implication true. No induction on A appears in this equivalence.

External induction on formula syntax, using binder congruence and H-composite, proves both interpretation composites. Numerical staged descriptions are identified with themselves; if one passes to a different conventional PR enumeration, literal-equation induction uniquely identifies the corresponding fixed recursions. Both internally definable domain isomorphisms have graph $x = y$. The progressive-binder calculus is therefore equiconsistent and whole-domain bi-interpretable with full first-order iPRA.

Limit of the reduction. This presentation has an unrestricted single HI rule, but it pays for that uniformity with explicit BoolCases and a distinction between Step and literal-equation Iter. No theorem here makes Cases redundant, and no smaller complete Python checker for this presentation has been established. It is therefore an alternative logical basis, not evidence that the measured frontier has been beaten.

7 Why merging the initializer into the update loses multivariate power

Attribution. The one-coordinate majorant obstruction, including the counter-aware recursor considered here, already appears in the current counter-congruence and transient-seed drafts. The following is a self-contained rederivation, not a new theorem claimed by this investigation. [5, 4]

The independent initializer in the numerical iterator looks like an attractive input to remove. The following obstruction rules out one particularly short proposed replacement as a direct primitive-recursive term basis. It does not rule out a different-domain or relational interpretation of that replacement.

7.1 The two candidate operations

The unary-update candidate has numerical terms generated by variables, zero, and $F(n; v.h)$, where only the fixed finite update body h binds v. Its proposed computation equations are

$$F(0; v.h) = S\big(h[0/v]\big), \qquad F(Sn; v.h) = h[F(n; v.h)/v]. \tag{F}$$

Here $S(a)$ is the abbreviation $F(0; v.a)$ with v absent from a. The counter-aware candidate instead has the constructor $F(n; u, v.h)$ and equations

$$F(0; u, v.h) = S\big(h[0/u, 0/v]\big), \qquad F(Sn; u, v.h) = h[n/u, F(n; u, v.h)/v]. \tag{F$^+$}$$

The two substitutions in the successor equation are simultaneous. Again $S(a) = F(0; u, v.a)$ with fresh u, v. Passive parameters of h are held fixed through the recursion.

For the obstruction, use the intended standard numerical interpretation: the seed is the ordinary successor of the displayed update-at-zero, and the successor clauses are ordinary primitive recursions. Every fixed term therefore denotes a primitive-recursive function. The issue is the converse: which primitive-recursive functions can be denoted?

Both proposals define the zero test

$$Z(n) = F(n; v.0), \qquad Z(0) = 1, \quad Z(Sn) = 0$$

(with an unused u in the counter-aware version). They also define

$$A(n, a) = F(n; v.a), \qquad A(0, a) = Sa, \quad A(Sn, a) = a.$$

Thus a monotonicity argument cannot exclude either proposal.

The counter-aware proposal even defines ordinary predecessor. Put

$$W(n) = F(n; u, v.u), \qquad P(n) = W\big(A(Z(n), W(n))\big).$$

Then $W(0) = 1$ and $W(Sn) = n$. Consequently

$$P(0) = W(A(1, 1)) = W(1) = 0,$$

whereas

$$P(Sn) = W(A(0, n)) = W(Sn) = n.$$

These endpoint computations need no induction. The obstacle is multivariate growth, even when these useful unary operations are present.

7.2 The single-coordinate majorant invariant

Theorem. For every term $t(x_1, \ldots, x_k)$ in either proposed language, one of the following holds in the standard numerical interpretation:

1. t is bounded by a fixed natural number; or
2. there is an index $i \leq k$ and a nondecreasing unary primitive-recursive function b such that

$$t(x_1, \ldots, x_k) \leq b(x_i) \quad \text{for every } (x_1, \ldots, x_k) \in \mathbb{N}^k. \tag{M}$$

The selected coordinate and the bounding function may depend on the fixed syntax of t, but do not depend on its numerical inputs.

Proof. Induct externally on the finite syntax of terms. A variable is bounded by the identity function of its own coordinate, and zero is bounded by zero. It suffices to treat F^+, because the unary-update case omits one of the cases below.

Write the recursion term as $F(n(\vec{x}); u, v.h(u, v, \vec{x}))$. By the induction hypothesis, h has a constant majorant or a nondecreasing unary majorant in one of $u, v, x_1, \ldots, x_k$.

If h is bounded by a constant c, every value of the recursion is at most $c + 1$. If h is bounded by $b(x_i)$ for a passive coordinate, its seed is at most $b(x_i) + 1$ and every successor value is at most $b(x_i)$. This gives the majorant $b(x_i) + 1$, independently of the counter and previous state.

If $h(u, v, \vec{x}) \leq b(u)$, the seed is at most $b(0) + 1$ and the value at stage $j + 1$ is at most $b(j)$. Thus the value at stage j is bounded by the nondecreasing unary function $b(j) + 1$.

Finally, suppose $h(u, v, \vec{x}) \leq b(v)$. Define by ordinary primitive recursion

$$c(0) = b(0) + 1, \qquad c(j + 1) = \max\{c(j), b(c(j))\}.$$

This is a nondecreasing unary primitive-recursive function. Its zero value bounds the seed. If the state at stage j is at most $c(j)$, monotonicity of b makes the next state at most $b(c(j)) \leq c(j + 1)$. Hence the recursion value at stage j is at most $c(j)$.

In each of the last two cases the result is bounded by a nondecreasing unary primitive-recursive function of the counter term $n(\vec{x})$. Apply the induction hypothesis to that term. A constant bound for n gives a constant bound for the whole recursion; a bound $n(\vec{x}) \leq d(x_i)$ gives the required bound by composition. This exhausts the cases and proves the theorem. □

7.3 Consequences and exact scope

Neither language denotes addition $a + b$. If a proposed addition term were bounded in a, holding $a = 0$ and increasing b would contradict the bound. A bound in b fails symmetrically, and a constant bound fails as well.

Neither language denotes a general numerical selector

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

A bound in the control coordinate fails by fixing $n = 0$ and increasing a. A bound in a fails by fixing $a = 0, n = 1$ and increasing b. A bound in b fails by fixing $b = 0, n = 0$ and increasing a. Additional dummy free coordinates do not help, because they can be fixed while the selected output grows.

The counter-aware predecessor construction therefore does not complete a primitive-recursive basis. The ordinary initialized iterator avoids the invariant precisely because its independent initializer can carry one arbitrarily large input while its iteration counter carries another: $I(b, a; v.Sv) = a + b$.

This theorem excludes a direct compilation of all primitive-recursive functions into terms with their usual numerical meanings. It does not show that no stronger proof system over the reduced syntax could interpret arithmetic relationally, that no other domain could support a bi-interpretation, or that the existing initialized basis is absolutely minimal. No such broader impossibility claim is used here.

8 Why atomic case introduction cannot simply be removed

Attribution. The counterinterpretation and the lost theorem $D_{x,y}(y = 1)$ already appear in the current flat-computation draft. The following is a self-contained rederivation of that inherited obstruction, not a new contribution. [2]

Another tempting reduction is to make the introduction mode automatic: use progress when the matrix is a literal equation, and case introduction when it is compound. This would remove one certificate choice. It does not preserve the native theory with its existing elimination rule.

Recall that $\mathsf{Q}_y B(y)$ is an internal clause with meaning $B(0) \to \forall y B(y)$, and that paired elimination of $\mathsf{D}_{x,y} B(x, y)$ supplies such a clause at a fresh witness. The unrestricted introduction inventory allows

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}$$

for every matrix, and

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)}$$

for literal equation matrices. Both rules have the usual freshness conditions. The proposed reduction deletes the first rule exactly when B is a literal equation.

A sound alternative semantics for the reduced rules. Take the standard natural numbers for all numerical terms and actual equality for equations. Interpret Q as above. Interpret D recursively by

$$(\mathsf{D}_{x,y} B)^\dagger = \begin{cases} \exists x \, \forall y \, (B^\dagger(x, y) \to B^\dagger(x, Sy)), & B \text{ is a literal equation,} \\ \exists x \, (B^\dagger(x, 0) \to \forall y B^\dagger(x, y)), & B \text{ is compound.} \end{cases}$$

This is a definition on each fixed finite formula, not a truth predicate inside the calculus.

Forced progress introduction is sound by its direct meaning. Compound case introduction is sound by the zero-or-successor property of the standard natural numbers. For paired elimination with an atomic matrix, ordinary numerical induction turns the supplied progressiveness into its conditional universal closure. Paired elimination with a compound matrix and closure specialization are sound directly. Equality transport is sound because numerical substitution leaves the distinction between a literal equation and a compound matrix unchanged. Numerical computation and every context-availability operation are sound as well. Therefore every theorem of the reduced rule system is true under $\dagger$.

A lost theorem. Let $1 = S0$ and let the witness coordinate be unused. The original calculus proves

$$\mathsf{D}_{x,y}(y = 1).$$

Indeed, case introduction assumes its zero instance $0 = 1$; numerical explosion yields $Sz = 1$, so the rule applies. Numerical explosion here only uses the existing numerical selector and equality transport: an assumption $0 = 1$ identifies its two endpoints and hence yields any literal equation. No induction on a compound formula is required.

Under †, however, this displayed formula says

$$\exists x \, \forall y \, (y = 1 \to Sy = 1),$$

which is false at $y = 1$. Consequently it is not derivable in the reduced calculus.

The padding equivalence between A and a suitably gated paired closure does not repair this counterexample. Padding can move a proof obligation into a compound matrix, but the existing elimination interface only returns conditional closure; it does not recover the progressiveness required to introduce an arbitrary raw atomic paired formula. Thus removing atomic case introduction would require a substantive change of rules and a new interpretation-composite proof. It is not an admissible simplification of the present kernel.

Attribution and the full inherited equivalence proof

The next four appendices are reproduced from the current `ipra_flat_computation_2026-11-10.zip`, with its attribution to earlier drafts retained. They establish the numerical and logical bridge for the theory implemented by the two new checkers. They are not new contributions of this investigation. Their original bytes, member paths, and hashes are recorded in `evidence/provenance.json`. Some historical descriptions of "new" changes in the first appendix refer to that earlier work. The progressive-binder presentation uses the same ordinary completion theorem, with counter-aware primitive recursion in place of the scalar compiler.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [10] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N(I(Sy, 2E(x); H)) \\ &= N(I(y, H(2E(x)); H)) \\ &= N(I(y, 3E(x); H)) \\ &= N(3I(y, E(x); H)) \\ &= N(I(y, E(x); H)) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

C From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [11]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix B, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

C.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion.

In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

C.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

C.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

C.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}(K(d(Sa, b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, (a = b \vee (a = b \to \bot)) \tag{41}$$

is a theorem, not an extra hypothesis.

C.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

C.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

D Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [12]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

D.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

D.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

D.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

E Complete measured programs

The following are entire trusted modules, including JSON input and the public verdict protocol. No producer, measurement utility, arithmetical evaluator, or theorem library is imported.

E.1 Iterative checker with immutable contexts

```
from json import load
k = 0
def i(a, t=None, d=0):
    try:
        return [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    while i(a) not in g:
        m, s, u, t = q()
        if m:
            c(m, g)
            if m in ([s, u], not s and [t]) and i(t, u) == a:
                a = i(t, s)
            h = m[2:]
        elif m is None:
            h = [[s, u, t], i(t, [list(*s), u, t], 1) if s else u]
        else:
            h = a = i(a[2], t, 2)
        k -= 1
        g += i(h, k if m or len(a) % 2 == m else 0),
        a = i(a, [k])
try:
    q = load(open(0)).pop
    c(q(), ())
except:
    print('REJECT')
    raise
print('ACCEPT')
```

E.2 Compact byte profile

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if a * 0 == [] else a
     if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = m in ([s, u], not s and [t]) and i(t, u) == a and i(t, s) or a
        h = m[2:]
    elif m is None:
        h = [[s, u, t], i(t, [list(*s), u, t], 1) if s else u]
    else:
        h = a = i(a[2], t, 2)
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == m) and k)])
try:
    c((q := json.load(open(0)).pop)(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, current draft collection*, commit `0300a290bedee5019c974291850b798378438b2e`. https://github.com/VladimirReshetnikov/ProveIt/tree/0300a290bedee5019c974291850b798378438b2e/docs/drafts.

[2] *Flat computation certificates for intuitionistic primitive recursive arithmetic*, `ipra_flat_computation_2026-11-10.zip` in [1]. Direct source of the implemented native theory, packet format, regression corpus, and attributed proof appendices.

[3] *Constructing computation sources for iPRA*, `ipra_constructed_computation_2026-11-10.zip`, in [1]. Source of the 277-token comparison program.

[4] *A one-state recursor for iPRA*, `ipra_transient_seed_2026-11-10.zip`, in [1]. Source of the 311-node comparison program.

[5] *Counter congruence suffices for iPRA*, `ipra_counter_congruence_2026-11-10.zip`, in [1]. Source of the complete 672-byte ancillary comparison program.

[6] *Local rewriting and successor-seeded scalar recursion for iPRA*, `ipra_local_rewriting_2026-11-10.zip`, in [1]. Source of the 628 raw-byte comparison program; its normalized size is 685 bytes.

[7] *Numerical axioms as local assumptions*, `ipra_generated_assumptions_2026-11-10.zip`, in [1]. Source of the known-witness assumption argument and the inherited arithmetic proof chain reproduced by [2].

[8] *Naming values before specialization*, `ipra_named_specialization_2026-11-10.zip`, in [1].

[9] *One Assumption at a Time: A Smaller iPRA Proof Kernel*. Choice-DI development reproduced with attribution in [8, 7, 2].

[10] Inherited scalar predecessor development, preserved as `reference/scalar_predecessor.tex` in [2].

[11] Inherited forward-left equality-reflection development, preserved as `reference/equality_reflection.tex` in [2].

[12] Inherited progress-closure pairing and primitive-recursion compiler, preserved as `reference/primitive_recursion.tex` in [2].

[13] Jeremy Avigad. *Proof mining*, lecture slides (2004), especially slides 12–13 on primitive recursive functions and arithmetic. https://www.andrew.cmu.edu/user/avigad/Talks/asl04.pdf.

[14] Inherited finite-parameter automorphism obstruction for unary arithmetic, preserved in the embedded `ipra_all_starts_2026-11-10.zip`, `reference/amalgam_original.tex` and `reference/fixed_seeds_original.tex`, within [1]. Exact copies and nested archive provenance are supplied in `reference/audit/prior_math/`.

[15] Inherited product-model independence of numerical coverage, preserved in the embedded `ipra_three_schema_2026-11-09.zip`, `report.tex`, within the history of [1]. The exact archived member identity is recorded in `reference/audit/prior_math/`.